DP Computer Science: A 3.3.5 Database Views (HL Only)

---

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Define and explain the concept of a **database view**
- Distinguish between **virtual views** and **materialized (snapshot) views**
- Describe the **advantages** of using database views, including:
  - Hiding data complexity
  - Ensuring data consistency
  - Providing data independence
  - Enhancing performance
  - Simplifying queries
  - Offering read-only or updatable data access
  - Improving security
- Identify scenarios where different types of database views might be most appropriate
- Construct basic SQL statements to create and use **virtual views** in DB Browser for SQLite

---

Key Vocabulary

Term	Definition
Database view	A virtual table based on the result of a stored query; it does not store data physically
Virtual view	A view that is not physically stored; the underlying query is executed each time the view is accessed
Materialized view	A view that stores the query results physically and must be refreshed periodically

Term	Definition
Snapshot	Another name for a materialized view—a point-in-time copy of data
Data abstraction	Hiding complex database structures from users
Data independence	Protecting applications from changes to the underlying database structure
Query simplification	Reducing complex queries into simple view references
Updatable view	A view that allows INSERT, UPDATE, and DELETE operations

---

Relevant ATL Skills

Skill	Description
Thinking	Understanding, application, analysis
Communication	Explaining concepts; using appropriate terminology
Self-Management	Organisation; metacognition (reflecting on understanding)

---

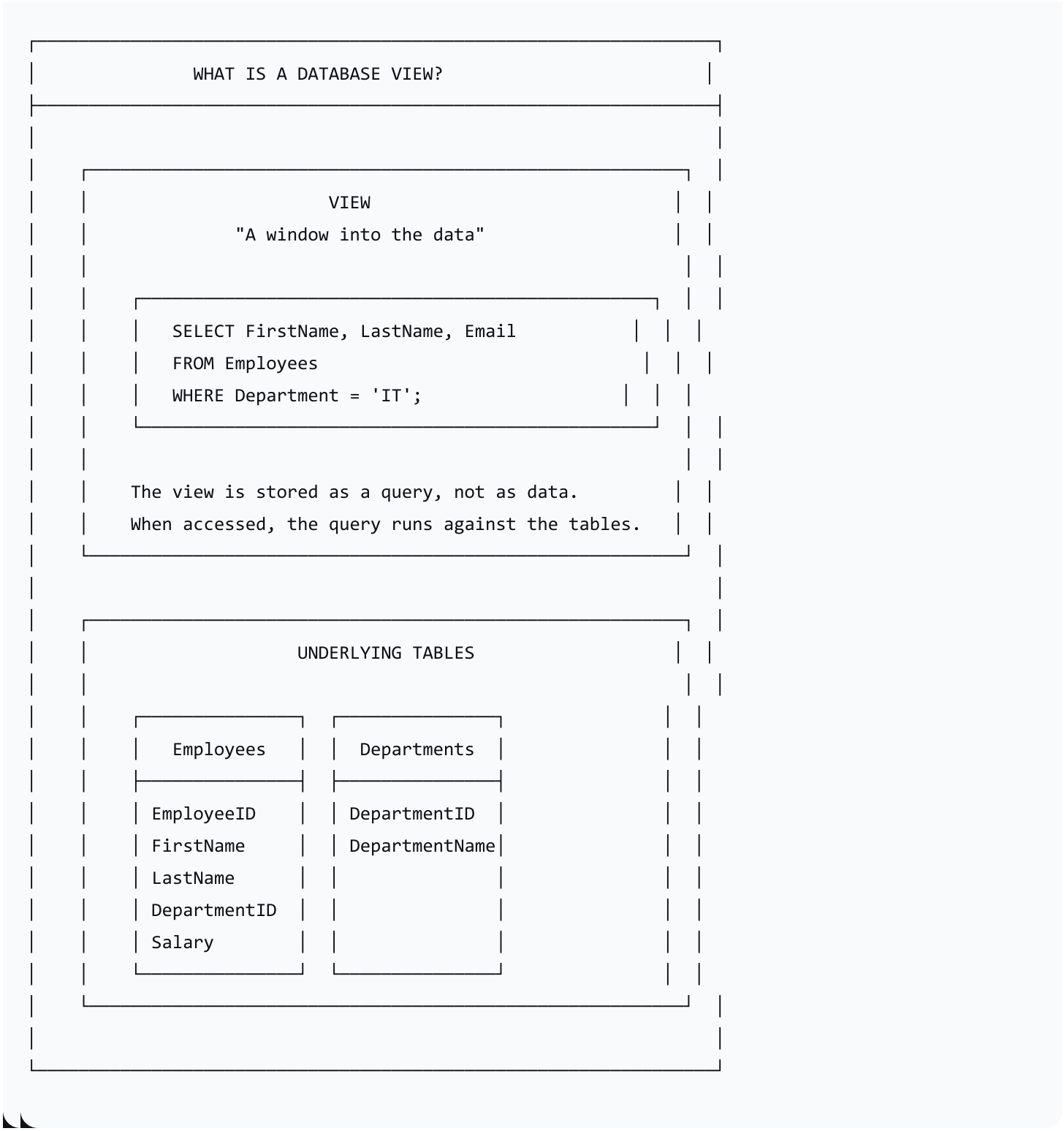
2. What is a Database View?

Definition

> **A database view is a virtual table based on the result of a stored query. It does not store data physically—it provides a dynamic, logical perspective of the underlying data.**

The View Concept

```
text
```



The Window Analogy

> A view is like a window into the database. The window shows you specific data in a specific format, but it is not the data itself—it's just a way of looking at it.

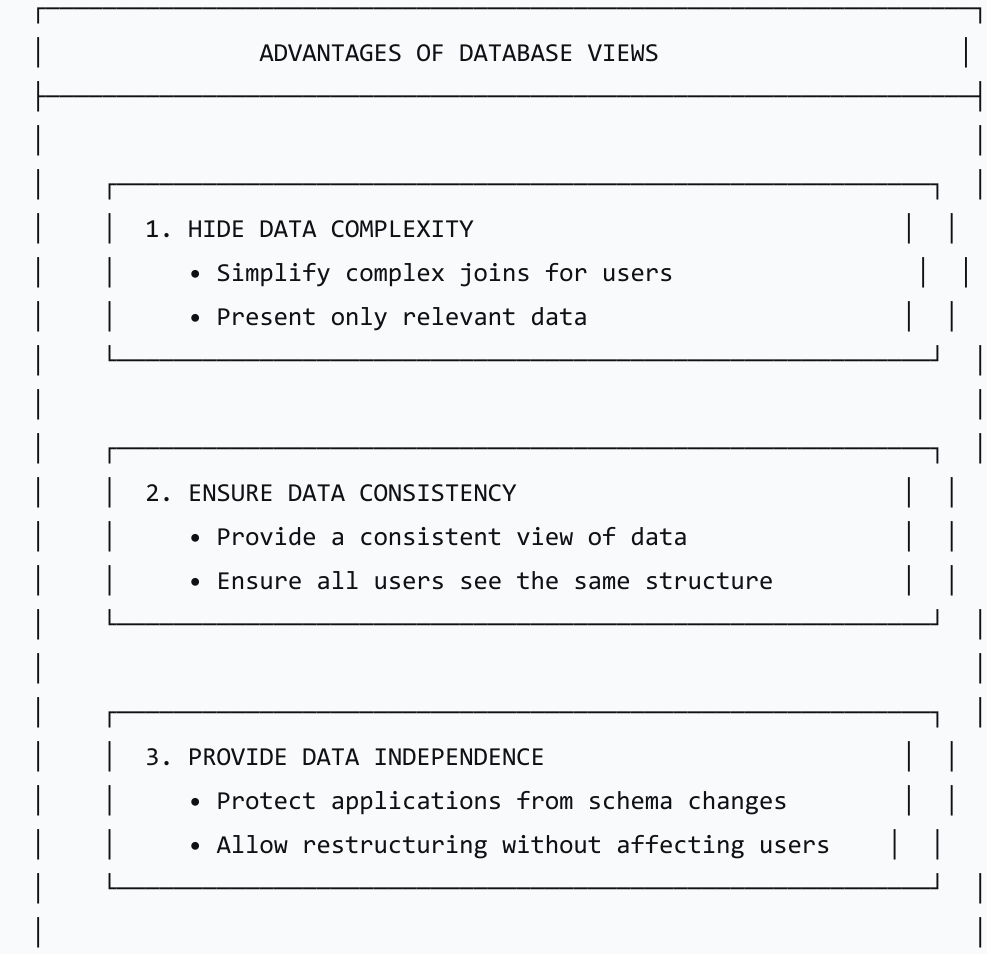
Aspect	Analogy
View	A window looking into a room
Underlying Tables	The objects in the room
Query	The angle and focus of the window
Virtual View	The window is always open; you see the current state
Materialized View	A photograph of what the window shows (stored image)

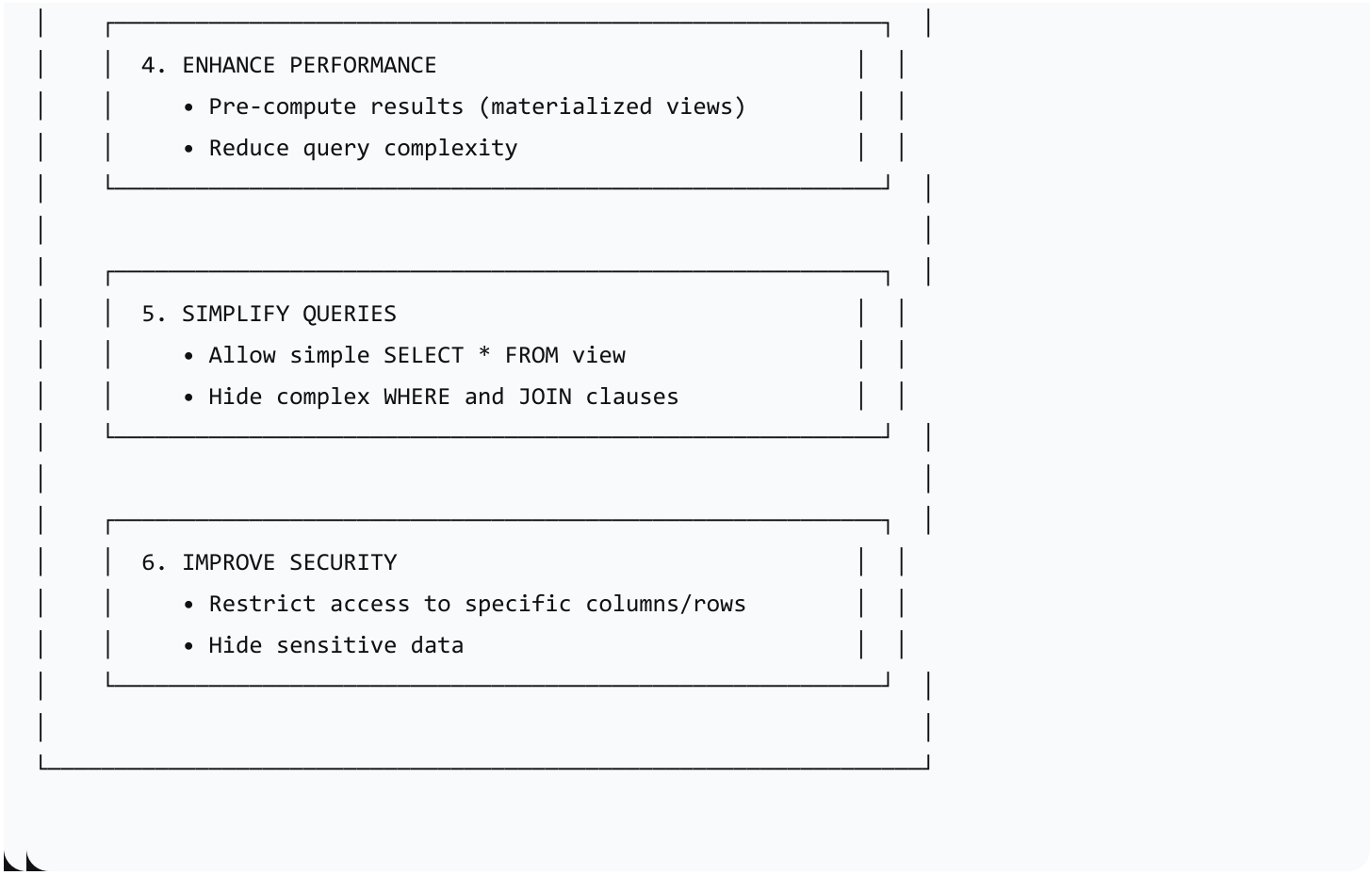
---

3. Advantages of Using Database Views

Summary of Benefits

```
text
```





Detailed Benefits

Benefit	Description	Example
Hide Complexity	Simplify complex queries for end-users	Join across 5 tables → a single view
Data Consistency	Ensure all users see data in the same way	All reports use the same view definition
Data Independence	Change table structure without breaking applications	Add a new column; views unaffected
Performance	Speed up queries (materialized views)	Pre-calculated totals for quick reports
Query Simplification	Reduce long, complex queries to simple SELECTs	`SELECT * FROM EmployeeReport`

Benefit	Description	Example
Security	Restrict access to sensitive data	Show employees' names but hide salaries
Read-Only Access	Provide controlled access without write permissions	Users can view but not modify data

4. Types of Database Views

Virtual Views

Aspect	Description
Definition	A view that is not physically stored; the query is executed each time the view is accessed
Storage	Stores only the query definition, not the data
Data Freshness	Always current (shows real-time data)
Performance	Query runs each time the view is accessed
Updateable	Can be updateable if based on a single table
Best For	Frequently changing data; real-time reporting

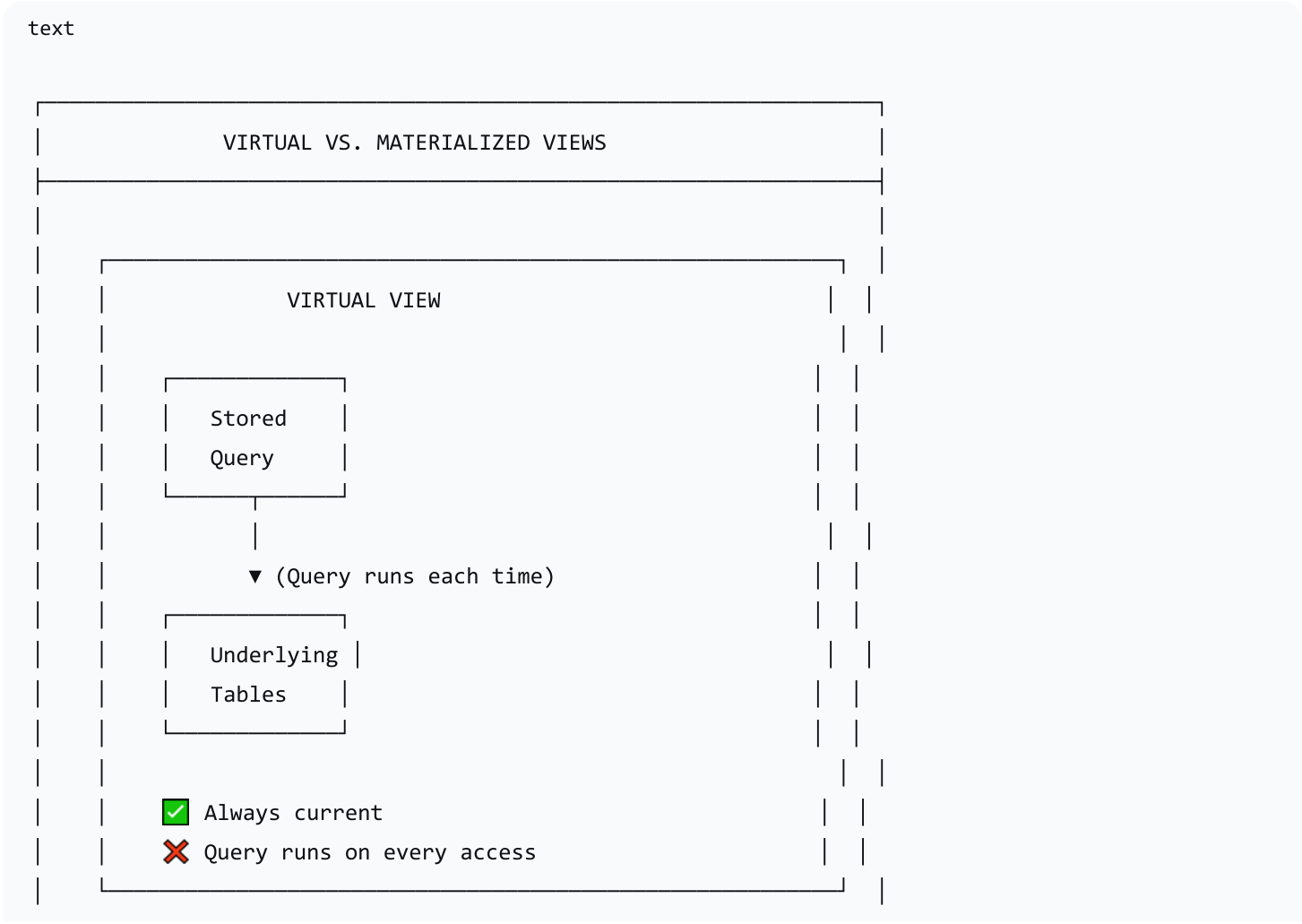
Virtual View SQL Example:

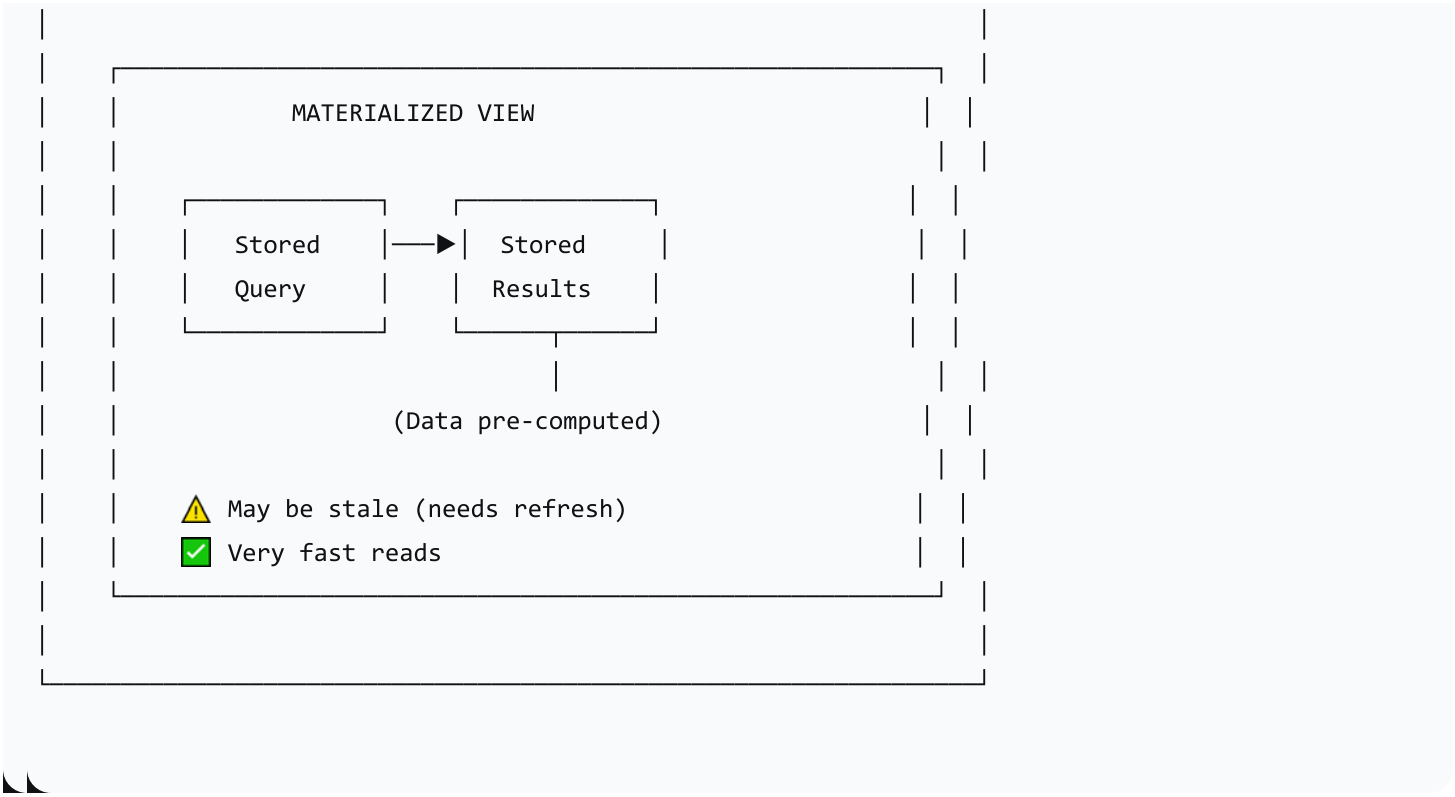
```sql

CREATE VIEW EmployeeITView AS
SELECT EmployeeID, FirstName, LastName, Email
FROM Employees
WHERE DepartmentID = 1;
```

Materialized Views (HL Only)

Aspect	Description
Definition	A view that stores the query results physically; must be refreshed periodically
Storage	Stores both the query definition AND the resulting data
Data Freshness	Stale until refreshed (point-in-time snapshot)
Performance	Very fast reads (data is pre-computed)
Updateable	Usually not updateable
Best For	Static or slowly changing data; analytical queries

Note: SQLite does not support materialized views natively, but some databases (Oracle, PostgreSQL) do.





5. Virtual vs. Materialized Views: Comparison

Feature	Virtual View	Materialized View
Storage	Stores only the query definition	Stores query results physically
Data Freshness	Always current	Stale until refreshed
Query Speed	Query runs on access (potentially slower)	Very fast reads (pre-computed)
Data Size	Minimal storage (just the query)	Significant storage (stores all data)
Updateable	Often updateable	Usually not updateable
Maintenance	Minimal (no refresh needed)	Requires periodic refresh
Best For	Real-time data, frequently changing	Analytical queries, static data
Resource Use	Low storage; CPU at query time	High storage; CPU at refresh time

Feature	Virtual View	Materialized View
Example Use	Current employee list	Monthly sales report

6. When to Use Which Type?

Virtual Views — Use When

Scenario	Why?
Real-time data is required	Always shows current data
Frequently changing data	No need to refresh
Data is small/medium size	Query runs quickly
Simple queries	No performance benefit from materializing
Data needs to be up-to-date	Always reflects current state

Materialized Views — Use When

Scenario	Why?
Heavy analytical queries	Pre-computed data is much faster
Data changes infrequently	Refresh can be scheduled
Complex aggregations	SUM, AVG, GROUP BY are pre-computed
Large datasets	Pre-computing saves time
Reporting dashboards	Quick response time for visualisations

7. Creating and Using Virtual Views

Syntax

```sql
CREATE VIEW view_name AS
SELECT columns
FROM tables
WHERE conditions;
```

Example: Creating a View

Database Schema:

```sql
-- Underlying tables
CREATE TABLE Employees (
    EmployeeID INTEGER PRIMARY KEY,
    FirstName TEXT,
    LastName TEXT,
    DepartmentID INTEGER,
    Salary REAL
);

CREATE TABLE Departments (
    DepartmentID INTEGER PRIMARY KEY,
    DepartmentName TEXT,
    Location TEXT
);
```

Creating a View:

```sql
-- Create a view that joins Employees and Departments
CREATE VIEW EmployeeDepartmentView AS
SELECT
    Employees.EmployeeID,
    Employees.FirstName,
```

```
    Employees.LastName,
    Departments.DepartmentName,
    Departments.Location
FROM Employees
INNER JOIN Departments
ON Employees.DepartmentID = Departments.DepartmentID
WHERE Departments.DepartmentName = 'IT';
```

Using the View:

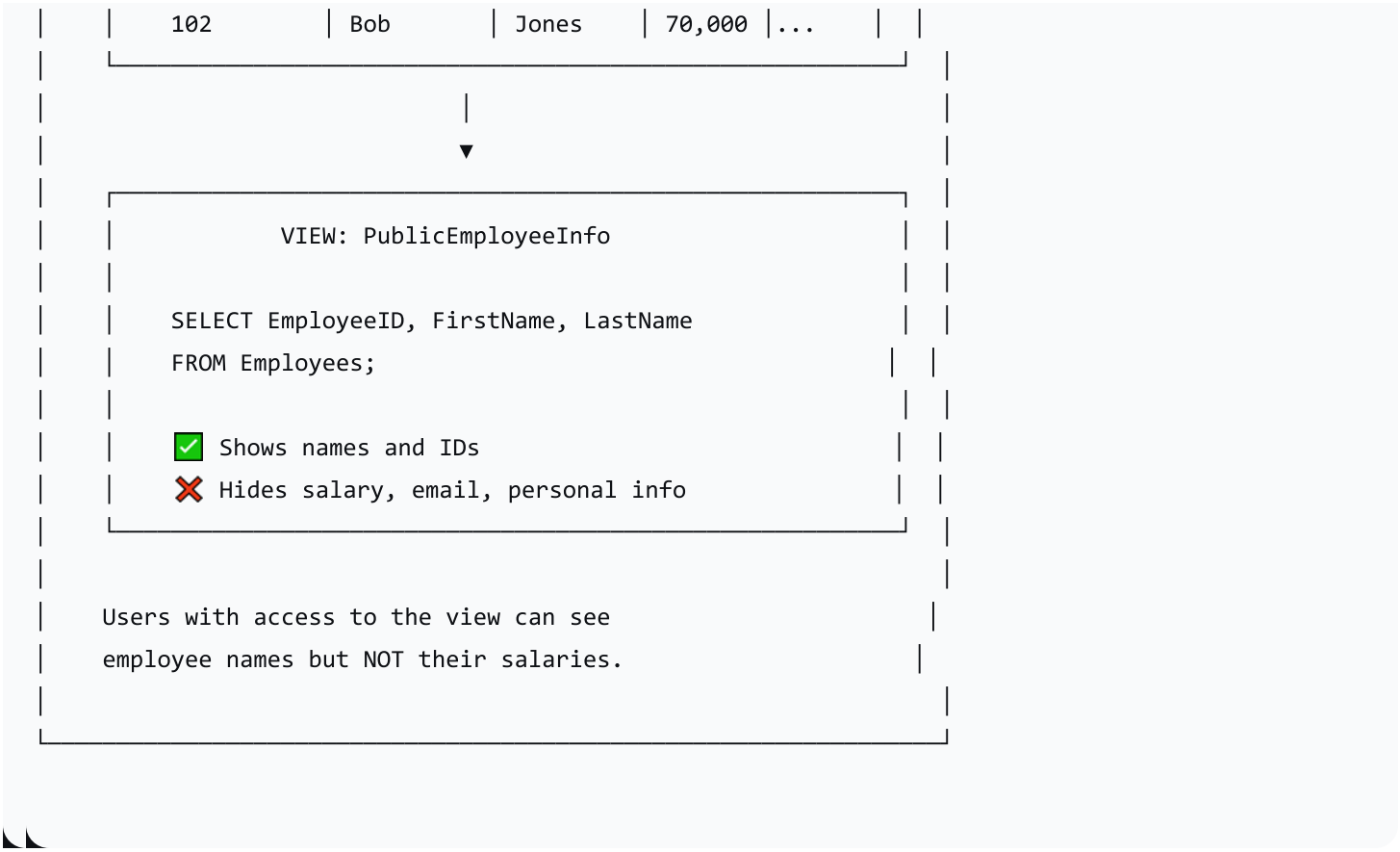
```sql
-- Using the view is as simple as selecting from a table!
SELECT * FROM EmployeeDepartmentView;

-- Filter the view
SELECT * FROM EmployeeDepartmentView
WHERE Location = 'New York';

-- Count employees from the view
SELECT COUNT(*) FROM EmployeeDepartmentView;
```

---

8. Security Benefits of Views

Restricting Access with Views

```text
┌─────────────────────────────────────────────────────────────┐
│              SECURITY WITH VIEWS                            │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│    ┌──────────────────────────────────────────────────┐     │
│    │              EMPLOYEES TABLE                     │     │
│    │                                                  │     │
│    │    EmployeeID | FirstName | LastName | Salary |...    │
│    │    ──────────┼──────────┼──────────┼────────┼────    │
│    │    101        | Alice     | Smith    | 85,000 |...    │
```



Updatable Views

A view can be updatable if it is based on a **single table** without:

- Aggregations (`SUM` , `AVG` , etc.)
- `GROUP BY` or `HAVING` clauses
- `DISTINCT` keyword
- Multiple tables (JOINs)

```sql
-- This view is likely updatable (single table, no aggregations)
CREATE VIEW EmployeeNames AS
SELECT EmployeeID, FirstName, LastName
FROM Employees;

-- Updates are allowed
UPDATE EmployeeNames
SET FirstName = 'Alicia'
WHERE EmployeeID = 101;
```

9. Real-World Use Cases

Use Case 1: HR Department

Requirement	Solution
Show employee names and departments	Create a view joining Employees and Departments
Hide salary information	Exclude Salary column from the view
Show employees by location	Add a WHERE clause for Location

Use Case 2: Sales Reporting

Requirement	Solution
Monthly sales summary	Materialized view with pre-computed totals
Complex aggregations	View with `SUM` and `GROUP BY`
Quick dashboard updates	Refresh materialized view nightly

Use Case 3: Application Development

Requirement	Solution
Multiple applications need the same data	Use a view to provide consistent data
Database restructuring	Keep views unchanged; update underlying tables
Legacy applications	Views can bridge old and new schemas

10. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Introducing Views	10 min	Define views; "window into the data" analogy; core concept
Benefits of Views	10 min	Hide complexity, consistency, independence, performance, query simplification, security
Virtual Views	10 min	Definition; how they work; CREATE VIEW example
Materialized Views	10 min	Definition; storage; refresh; performance benefits; trade-offs
Comparison	10 min	Virtual vs. Materialized; when to use which
Summary & Worksheet	10 min	Recap key concepts; introduce worksheet

---

Discussion Questions

Question	Purpose
"When would you use a view instead of writing the query directly?"	Understand benefits of views
"What is the difference between a virtual and a materialized view?"	Distinguish types
"Why would a company use views for security?"	Understand security benefits

Question	Purpose
"What are the trade-offs of using materialized views?"	Evaluate performance vs. storage

11. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Worksheet	Evaluate understanding of view concepts
Exit Ticket	Gauge understanding and identify areas needing clarification

Sample Exit Ticket Questions:

1. What is a database view?
2. What is the key difference between a virtual view and a materialized view?
3. Give one reason why views improve security.
4. When would you use a materialized view instead of a virtual view?

12. Differentiation

Level	Strategy
Support	Use the "window" analogy; provide visual aids; give clear examples
Challenge	Explore SQL syntax for creating views; research materialized view refresh strategies

13. Resources

- **Presentation** (Slide Deck)
- **eBook** (A3.3.5 with examples)
- **Worksheet:** View concepts and applications
- **Worksheet Answers**
- **DB Browser for SQLite**

---

14. Summary: Key Takeaways

Concept	Key Point
View	A virtual table based on a stored query
Virtual View	Query runs on access; always current
Materialized View	Stores results; very fast reads; needs refreshing
Benefits	Simplifies queries; improves security; provides data independence
Security	Restrict access to specific columns/rows
When to Use	Virtual = real-time; Materialized = analytical/reporting

```
text

┌──────────────────────────────────────────────────────────────┐
│                    KEY REMINDERS                             │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│    ✅ Views are NOT physically stored data                   │
│    ✅ Views are stored queries                               │
│    ✅ Virtual views = always current                         │
│    ✅ Materialized views = pre-computed, fast reads          │
│    ✅ Views can improve security and simplify queries        │
│    ❌ Materialized views need periodic refresh               │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

> *"Views are powerful tools for data abstraction—they protect users from complexity, shield applications from change, and provide a flexible security layer."*